A

A MACHINE LEARNING PROJECT REPORT

On

“HOUSE PRICE PREDICTION”

By

MISS. ISHITA URJIT PATIL

In Domain

DATA SCIENCE & MACHINE LEARNING

2026–2027

Technologies Used

PYTHON | GOOGLE COLAB | PANDAS | NUMPY | MATPLOTLIB | SCIKIT-LEARN

1. INTRODUCTION

House price prediction is an important application of Data Science and Machine Learning. The price of a property depends on several factors, such as its location, size, building type, construction year, overall condition, and available facilities.

The purpose of this project is to analyze housing data and develop a Machine Learning model that can estimate the selling price of a house based on its available features.

The project uses Python in Google Colab to perform data analysis, handle missing values, explore relationships between housing features and prices, visualize patterns, and train and evaluate prediction models.

By studying these factors, the project aims to understand which property characteristics are associated with higher or lower selling prices and how Machine Learning can support data-driven property price estimation.

1.1 Project Overview

- **Project Title:** House Price Prediction
- **Project Domain:** Data Science and Machine Learning
- **Programming Language:** Python
- **Development Environment:** Google Colab
- **Dataset:** HousePrediction.xlsx
- **Target Variable:** SalePrice
- **Project Type:** Regression — predicting a continuous numerical value.

1.2 Problem Statement

Estimating a house's selling price accurately can be challenging because property values depend on multiple factors. This project addresses the problem by analyzing historical housing data and building a Machine Learning model to predict house prices from relevant property features.

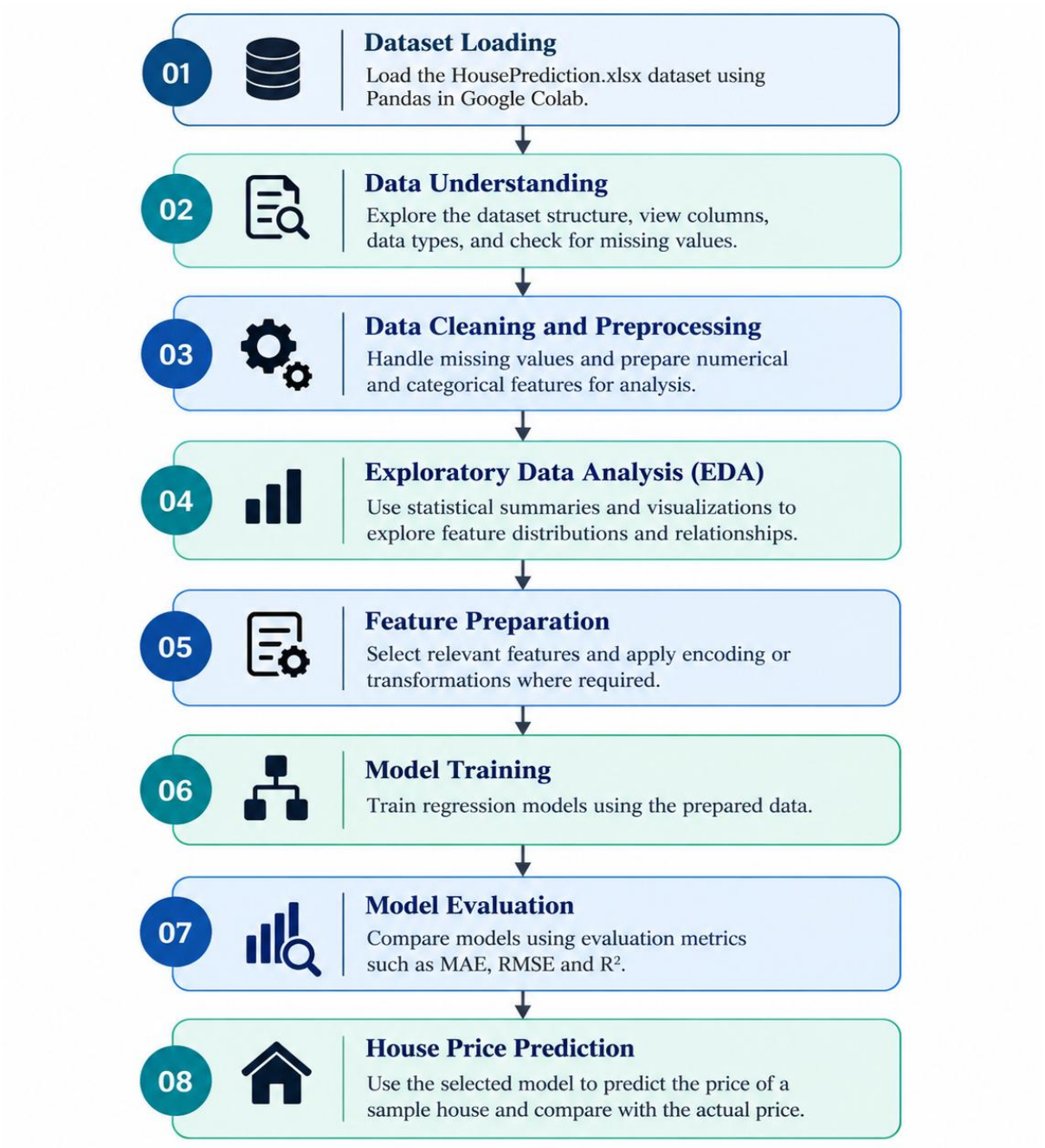
1.3 Project Objectives

1. Understand and analyze the housing dataset.
2. Identify and handle missing values appropriately.
3. Explore numerical and categorical features using statistical analysis and visualizations.
4. Study how house characteristics relate to selling prices.
5. Train and compare Machine Learning regression models.
6. Evaluate model performance using appropriate metrics.
7. Predict the selling price of a house using the selected model.

2. PROJECT METHODOLOGY

The project follows a systematic Machine Learning workflow to analyze housing data, prepare the dataset, identify important patterns, train prediction models, and evaluate their performance.

2.1 Workflow



2.2 Tools and Technologies

- **Python:** Programming language used for analysis and model development.
- **Google Colab:** Cloud-based environment used to execute Python code.
- **Pandas:** Used for loading, manipulating, and analyzing tabular data.
- **NumPy:** Used for numerical operations.
- **Matplotlib:** Used to create charts and visualizations.
- **Scikit-learn:** Used for data preprocessing, model training, and performance evaluation.
- **Microsoft Excel:** Used as the source format for the housing dataset.

3. REQUIREMENTS ANALYSIS

Requirements analysis identifies the software, tools, hardware, and data needed to develop and execute the House Price Prediction project.

3.1 Hardware Requirements

The project can be developed using a standard computer with internet access because Google Colab provides a cloud-based Python execution environment.

- **Processor:** Standard laptop or desktop processor
- **RAM:** 4 GB or above recommended
- **Storage:** Sufficient space for the dataset, notebook, and report
- **Internet:** Required to access Google Colab

3.2 Software Requirements

Python	Programming and machine learning model development
Google Colab	Writing and executing Python code in a cloud environment
Pandas	Loading, cleaning, and manipulating the dataset
NumPy	Numerical calculations and data operations
Matplotlib	Creating graphs and visualizations
Scikit-learn	Data preprocessing, regression model training, and evaluation
Microsoft Excel	Storing and viewing the source dataset
Microsoft Word	Preparing the project report

3.3 Dataset Requirements

Important dataset features include:

Id	Unique identifier for each property	Numerical / Identifier
MSSubClass	Dwelling type classification	Categorical
MSZoning	General zoning classification	Categorical
LotArea	Total lot area	Numerical
LotConfig	Configuration of the property lot	Categorical
BldgType	Type of building	Categorical
OverallCond	Overall condition rating of the property	Ordinal
YearBuilt	Original construction year	Numerical
YearRemodAdd	Year of remodeling or addition	Numerical
Exterior1st	Exterior covering material	Categorical
BsmtFinSF2	Type 2 finished basement area	Numerical
TotalBsmtSF	Total basement area	Numerical
SalePrice	Selling price of the house (target variable)	Numerical

3.4 Functional Requirements

The project should be able to:

1. Load and inspect the housing dataset.
2. Identify and handle missing values.
3. Analyze numerical and categorical features.
4. Generate visualizations to understand price patterns.
5. Train regression models using labelled housing data.
6. Evaluate model performance using appropriate metrics.
7. Predict the selling price of a sample house.

3.5 Non-Functional Requirements

- **Usability:** The notebook should be understandable and easy to execute.
- **Reliability:** Data preprocessing and model evaluation should be performed consistently.
- **Maintainability:** Code should be organized into clearly labelled sections.
- **Performance:** The model should generate predictions within a reasonable time.

4. DATASET DESCRIPTION AND DATA PREPROCESSING

4.1 Dataset Description

The project uses a dataset named **HousePrediction.xlsx** containing information about residential properties. The dataset has **2,919 rows and 13 columns**. It includes property characteristics and the selling price, which is used as the target variable for prediction.

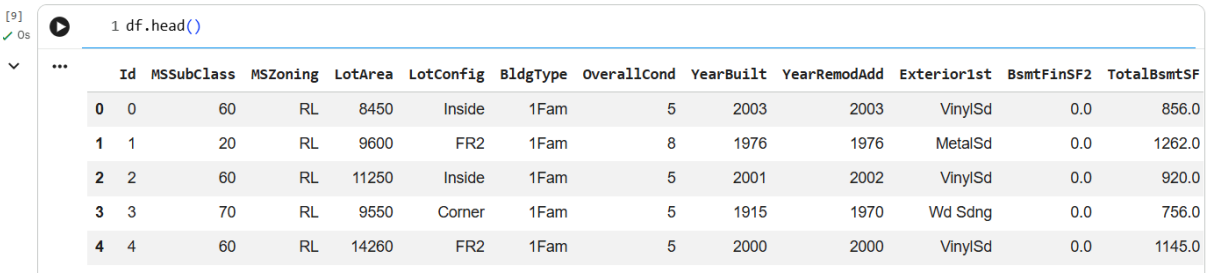
The dataset contains both numerical and categorical features. Numerical features represent measurable values, such as lot area and basement area, while categorical features describe property classifications, such as zoning and building type.



4.2 Target Variable

The target variable is **SalePrice**. It represents the selling price of a house. The machine learning model learns patterns between the available property features and their corresponding selling prices to estimate prices for other properties.

Since the target is a continuous numerical value, this is a **supervised regression problem**.



	Id	MSSubClass	MSZoning	LotArea	LotConfig	BldgType	OverallCond	YearBuilt	YearRemodAdd	Exterior1st	BsmtFinSF2	TotalBsmtSF
0	0	60	RL	8450	Inside	1Fam	5	2003	2003	VinylSd	0.0	856.0
1	1	20	RL	9600	FR2	1Fam	8	1976	1976	MetalSd	0.0	1262.0
2	2	60	RL	11250	Inside	1Fam	5	2001	2002	VinylSd	0.0	920.0
3	3	70	RL	9550	Corner	1Fam	5	1915	1970	Wd Sdng	0.0	756.0
4	4	60	RL	14260	FR2	1Fam	5	2000	2000	VinylSd	0.0	1145.0

4.3 Missing Value Analysis

Missing-value analysis was performed to identify incomplete data before model development. The dataset contains 2,919 rows, but SalePrice is available for only 1,460 rows. The remaining 1,459 rows have missing target values.

Rows with known selling prices can be used for supervised model training and evaluation. Rows without SalePrice cannot directly be used as labelled examples for training because their actual prices are unknown.

"Tell me how many missing values are present in every column."

```
[16]
✓ 0s
1 df.isnull().sum()
```

	0
Id	0
MSSubClass	0
MSZoning	4
LotArea	0
LotConfig	0
BldgType	0
OverallCond	0
YearBuilt	0
YearRemodAdd	0
Exterior1st	1
BsmtFinSF2	1
TotalBsmtSF	1
SalePrice	1459

only columns that have missing values

```
[17]
✓ 0s
1 missing_values = df.isnull().sum()
2
3 missing_values[missing_values > 0]
```

	0
MSZoning	4
Exterior1st	1
BsmtFinSF2	1
TotalBsmtSF	1
SalePrice	1459

dtype: int64

4.4 Data Preprocessing

Data preprocessing prepares the dataset for analysis and model building. The main activities include:

1. Inspecting the dataset structure, column names, and data types.
2. Checking missing values in the available features.
3. Identifying numerical and categorical columns.
4. Examining feature distributions and unusual values.
5. Preparing the required features for visualization and model training.
6. Applying suitable transformations or encoding where required by the model.

4.5 Importance of Data Preprocessing

Proper preprocessing improves data quality and helps prevent errors during model training. It also ensures that the selected features are in a suitable format for the machine learning algorithm.

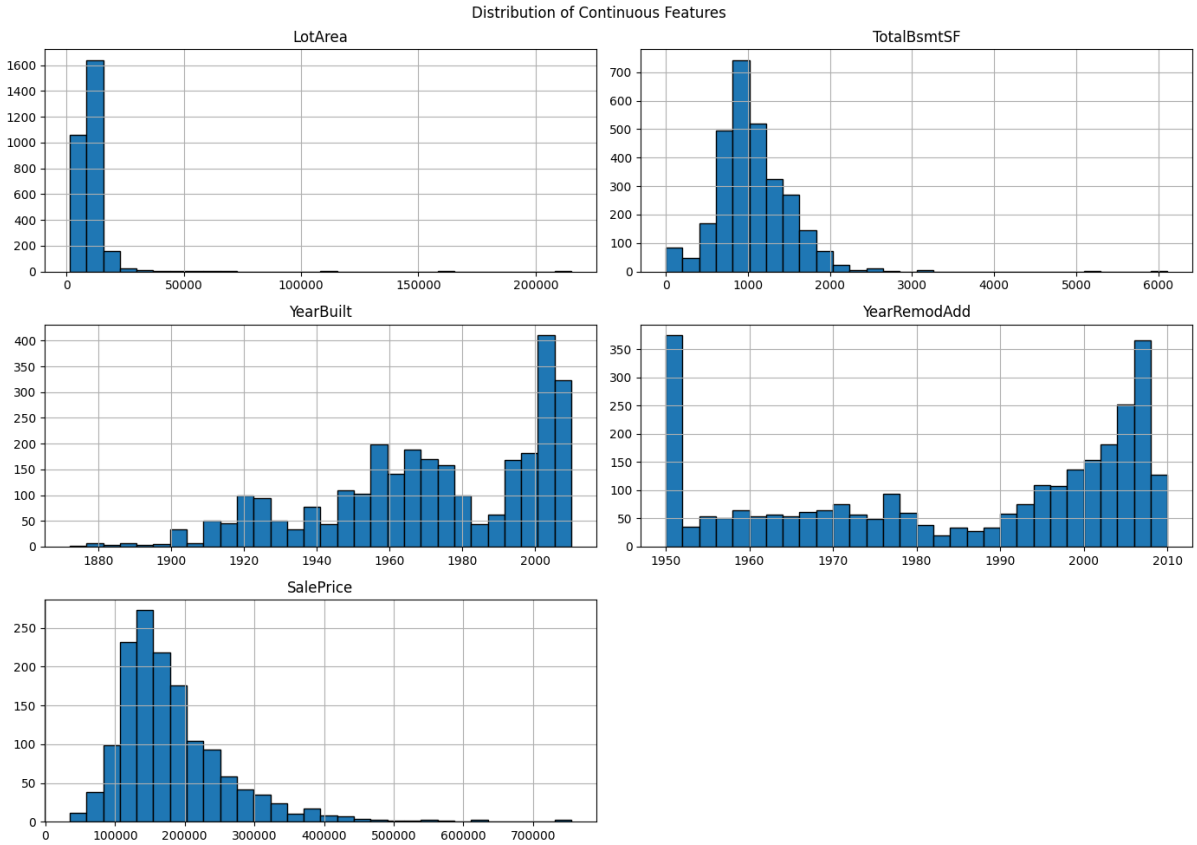
5. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is the process of examining a dataset using statistical summaries and visualizations to understand its structure, distributions, and relationships. In this project, EDA was performed to explore housing features and identify patterns related to house prices.

5.1 Analysis of Numerical Features

Numerical features such as LotArea, OverallCond, YearBuilt, YearRemodAdd, and TotalBsmtSF were examined to understand their distributions and possible relationships with the target variable.

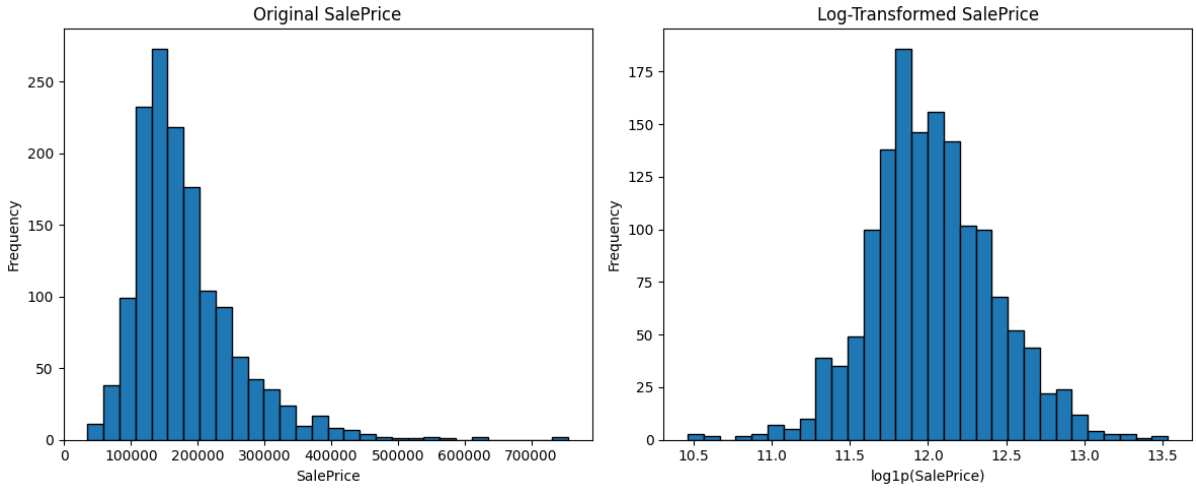
Histograms were used to visualize the distribution of numerical values. These plots helped identify how frequently different values occurred and whether a feature had a skewed distribution.



5.2 Analysis of SalePrice Distribution

The distribution of SalePrice was examined to understand the variation in house selling prices. A log transformation was also explored to make a highly skewed price distribution easier to analyze.

The transformed distribution can help reveal patterns that may be less visible in the original data. The target values must be converted back appropriately if predictions are made on a logarithmic scale.

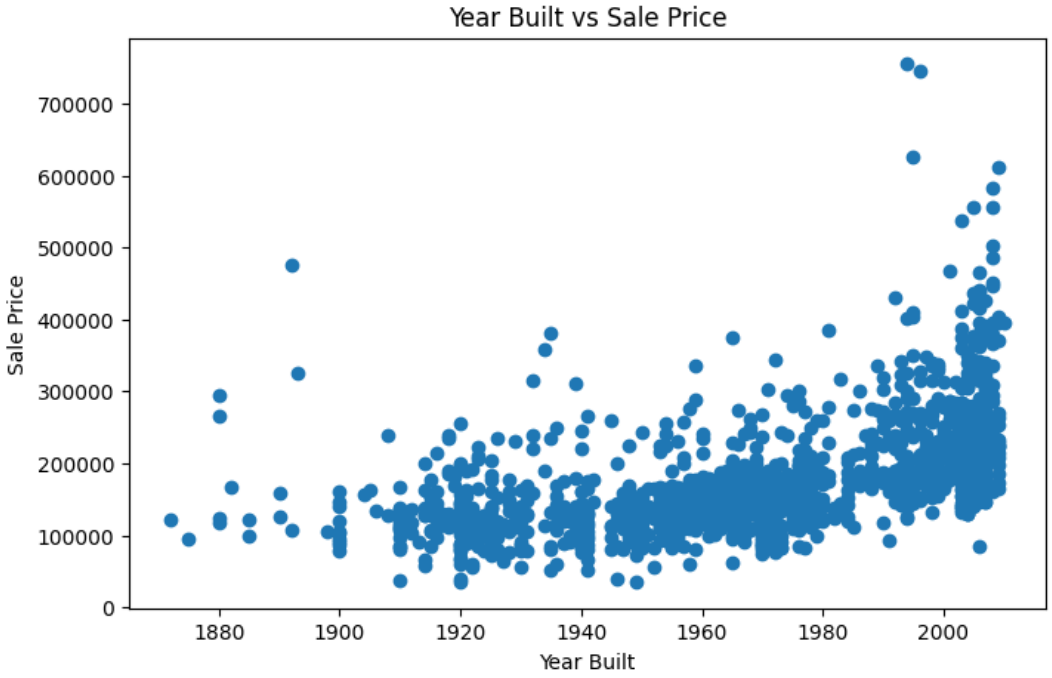


5.3 Relationship Between Construction Year and Sale Price

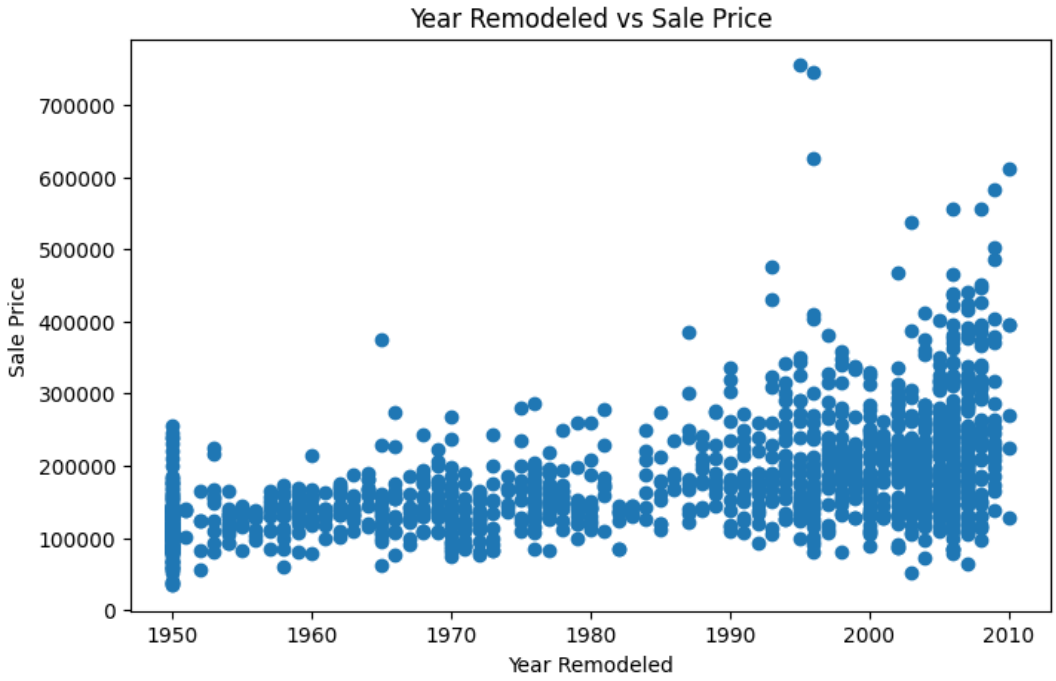
Scatter plots were created to examine the relationships between YearBuilt, YearRemodAdd, and SalePrice.

These visualizations help investigate whether the construction year or remodeling year is associated with differences in selling prices. They also make it easier to observe trends and variations among properties.

Year Built vs Sale Price



Year RemodAdd vs. Sale Price

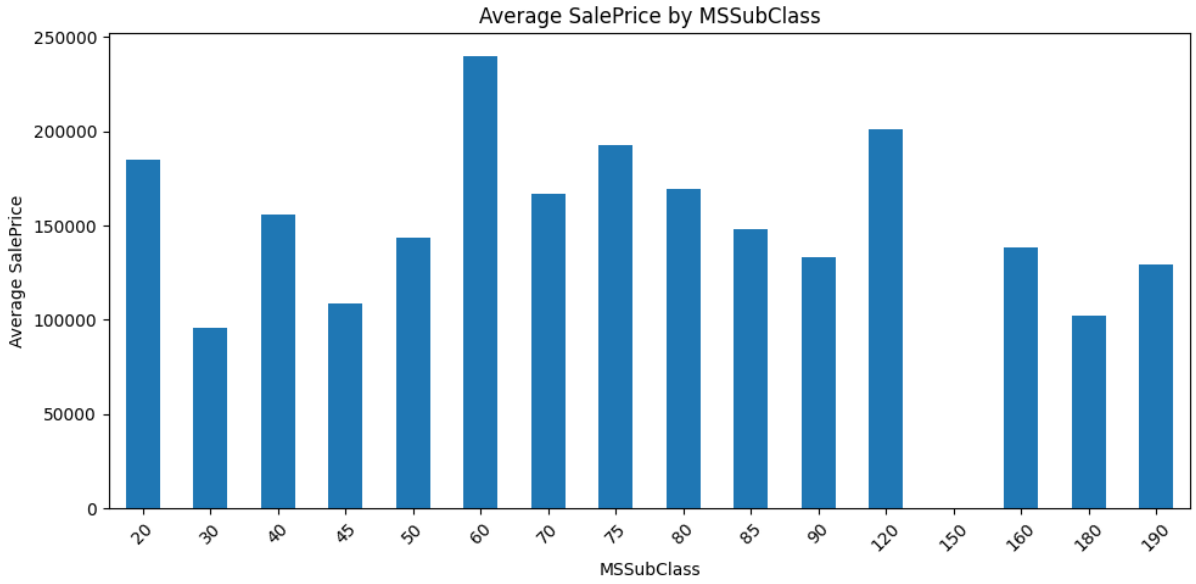


5.4 Analysis of Discrete Features

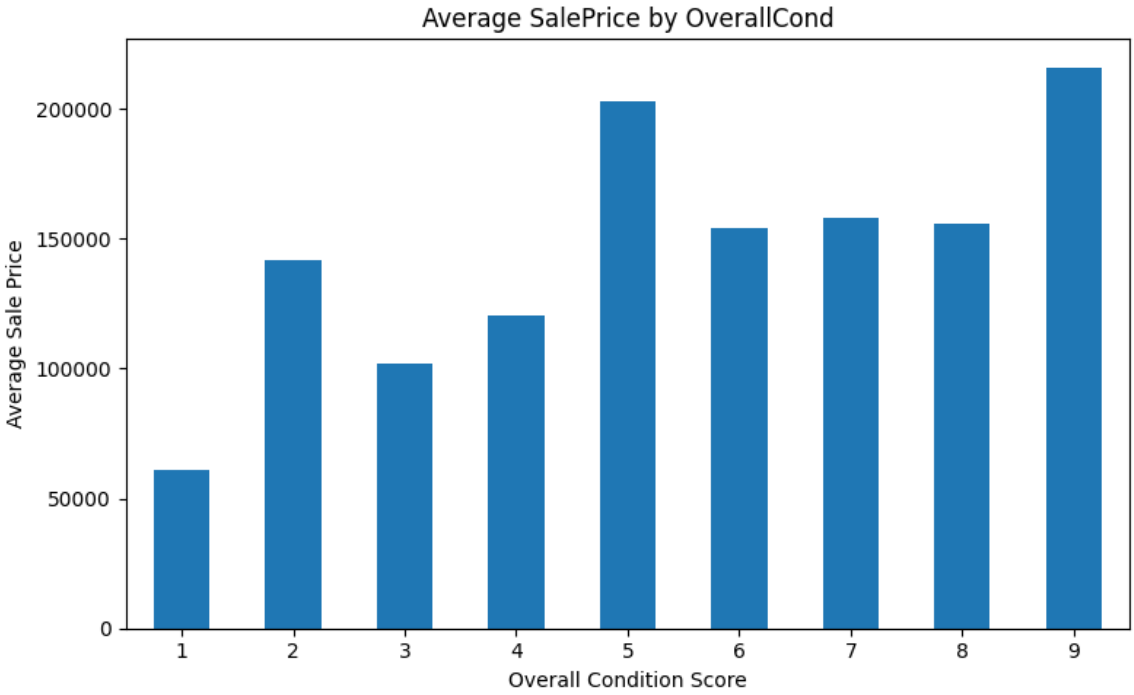
Discrete features such as MSSubClass and OverallCond were analyzed to understand how different categories or rating values were distributed.

A bar chart showing the average SalePrice for different MSSubClass values was used to compare average selling prices across dwelling classifications.

Average Sale Price by MS SubClass



Average Sale Price by Overall Cond

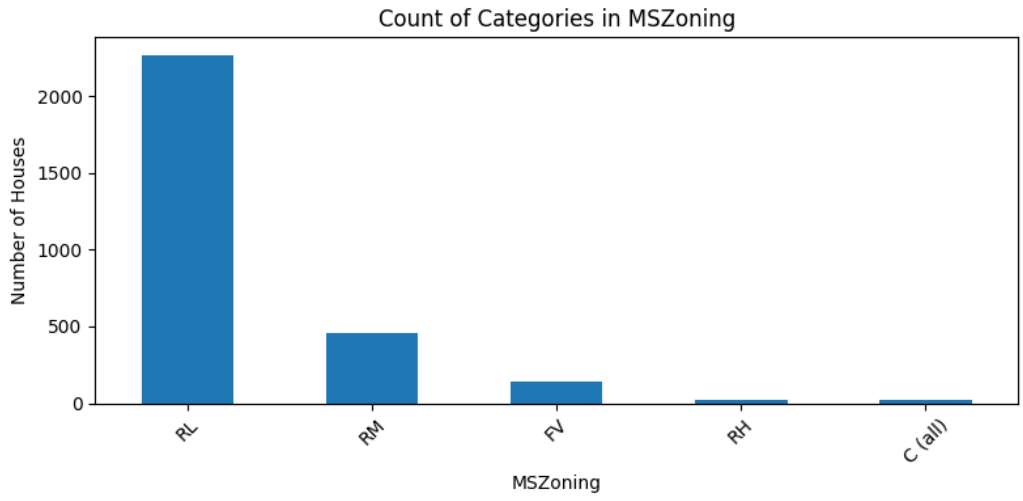


5.5 Analysis of Categorical Features

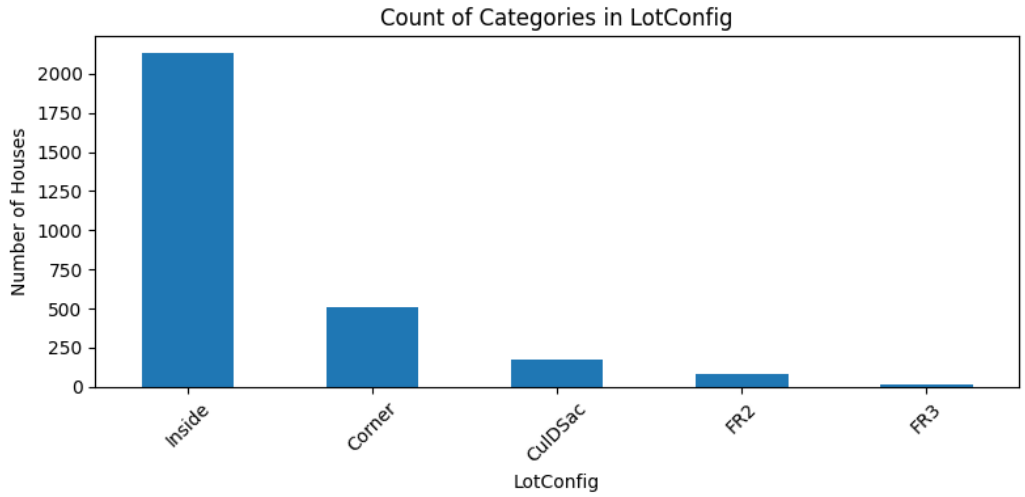
Categorical features, including MSZoning, LotConfig, and BldgType, were considered to explore differences between property groups.

These features help describe the type, location classification, and configuration of a property. Their relationship with selling price can provide useful information for selecting features for a prediction model.

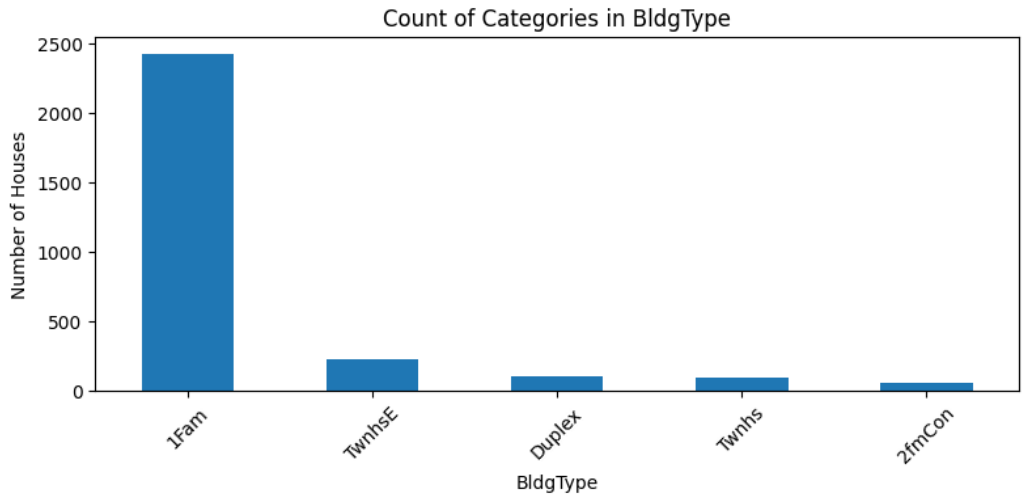
1. MSZoning Category Distribution



2. LotConfig Category Distribution



3. BldgType Category Distribution



5.6 Importance of EDA

EDA helped develop an understanding of the dataset before model training. It supported the identification of patterns, the examination of feature relationships, and the selection of relevant visualizations for further analysis.

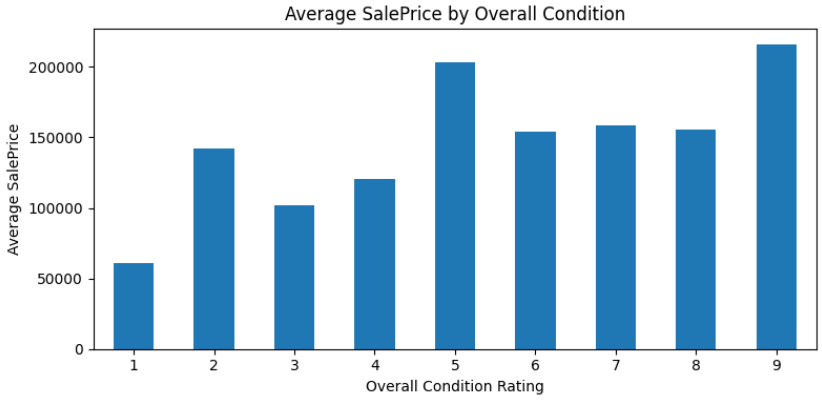
6. FEATURE ANALYSIS AND KEY OBSERVATIONS

Feature analysis helps identify the property characteristics that may be useful for predicting house prices. In this project, numerical and categorical features were examined to understand their potential relationship with SalePrice.

6.1 Overall Condition of the Property

The OverallCond feature represents the overall condition rating of a property. It was examined to understand how house prices vary across different condition ratings.

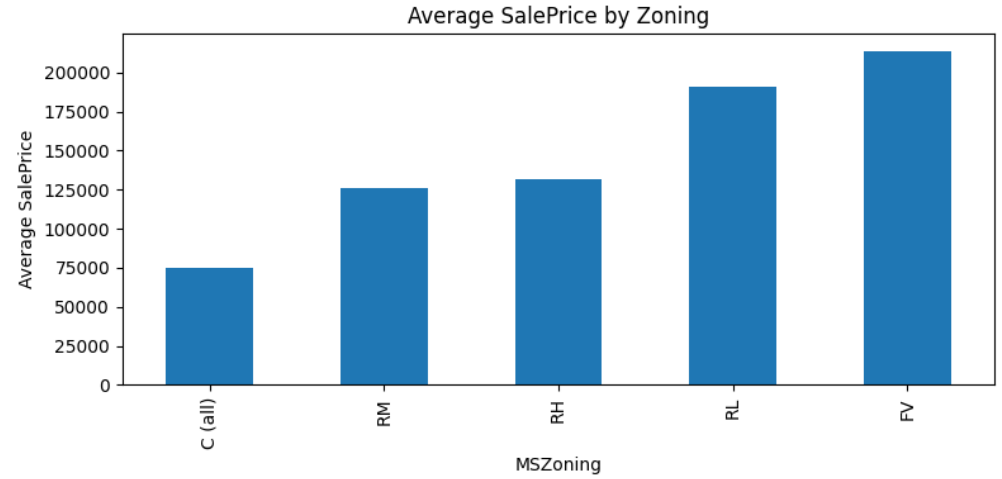
Comparing average selling prices across condition values can help investigate whether property condition is associated with price differences.

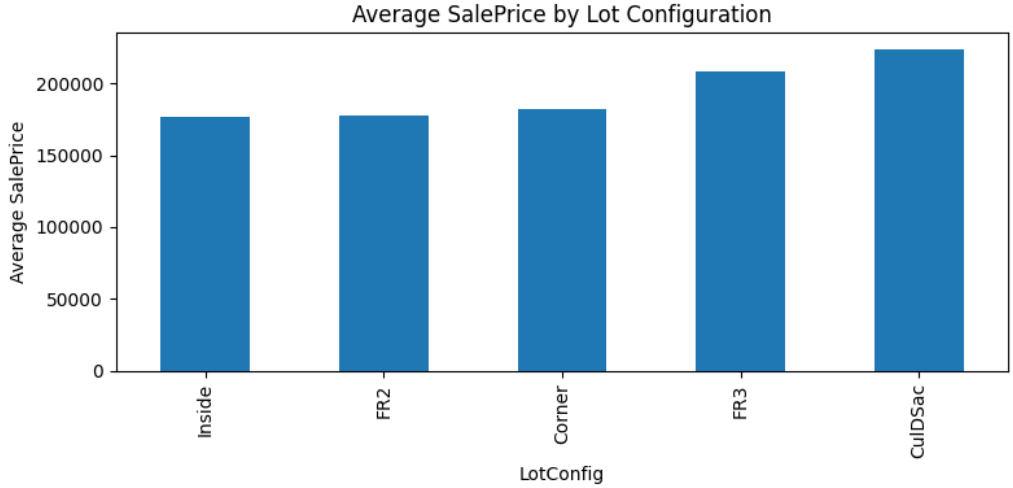


6.2 Zoning and Lot Configuration

The MSZoning feature describes the zoning classification, while LotConfig describes the configuration of the property lot.

These categorical features were considered to explore price differences between property groups. Their usefulness depends on the patterns present in the available labelled data.

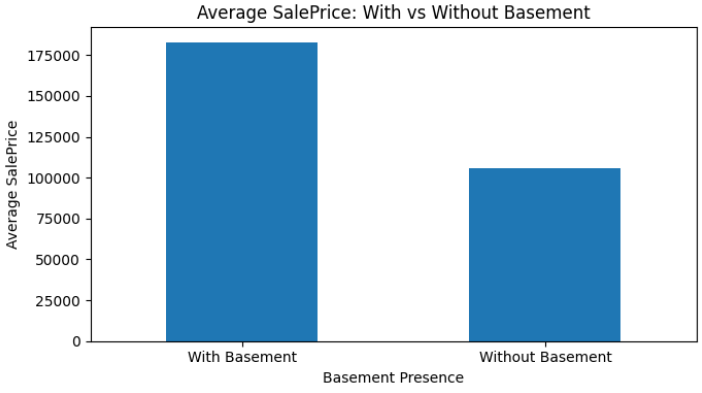




6.3 Basement Area and Availability

The TotalBsmtSF feature represents the total basement area in square feet. Basement-related information can help describe the size and characteristics of a property.

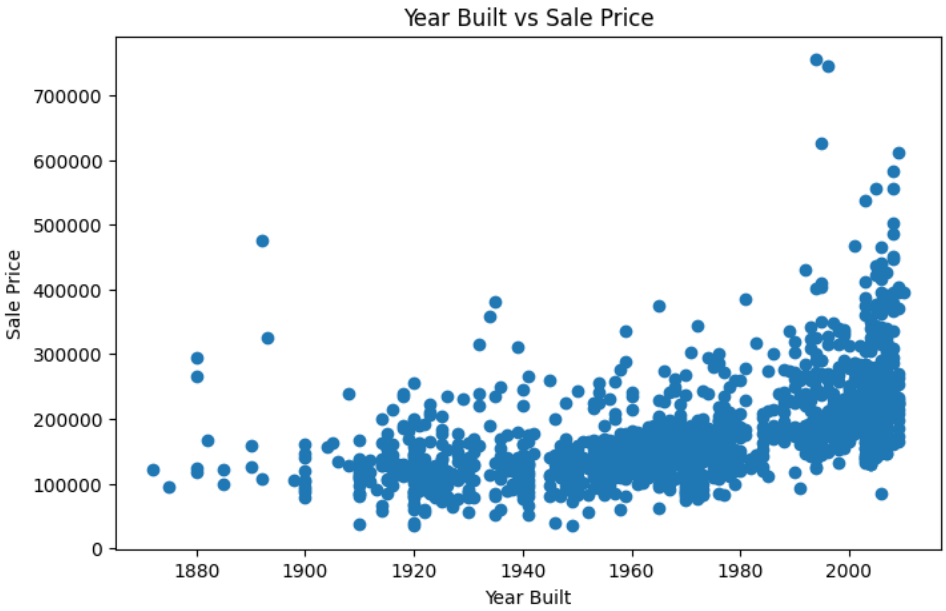
Examining basement area alongside selling price helps investigate whether differences in basement space are associated with differences in house prices. Missing values should be handled carefully so that missing information is not automatically treated as a genuine zero.

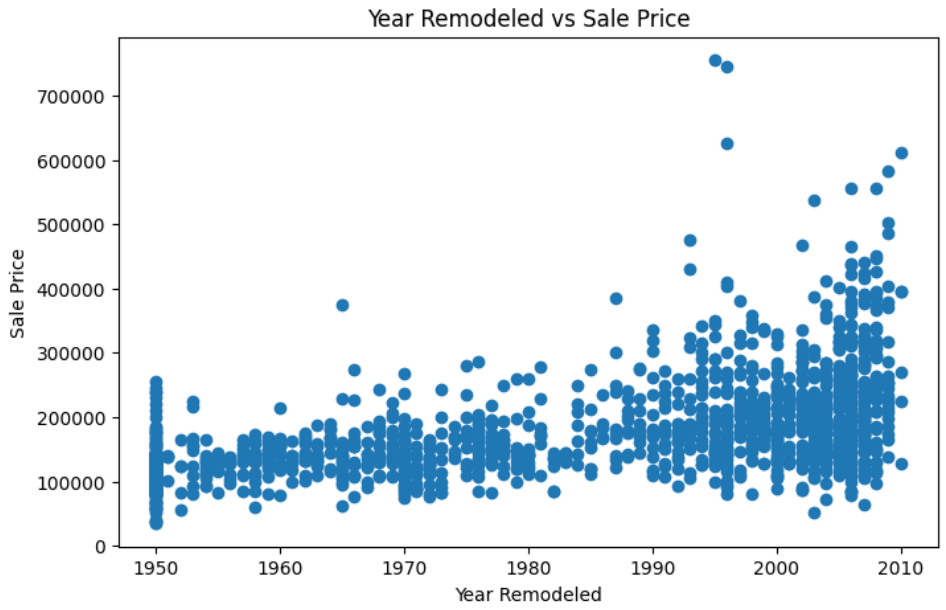


6.4 Construction and Remodeling Years

The YearBuilt and YearRemodAdd features represent the original construction year and the year of remodeling, respectively.

These features were explored using scatter plots to examine their relationships with SalePrice. They provide information about the age and renovation history of a property.





6.5 Categorical Features and Prediction

Categorical features describe property groups rather than continuous numerical measurements. They must be converted into a suitable numerical representation before being used by regression algorithms that require numerical inputs.

A categorical-only model was also examined to understand how well categorical information could predict house prices without relying on the numerical features.

6.6 Key Observations

The analysis highlighted the following points:

1. Housing data contains both numerical and categorical features.
2. SalePrice is the target variable used for supervised prediction.
3. Construction year, remodeling year, property condition, and basement area are useful features to investigate.
4. Categorical information can contribute to prediction, but its predictive performance needs to be evaluated.

7. MACHINE LEARNING MODEL DEVELOPMENT

Machine learning model development is an important stage of the House Price Prediction project. In this stage, the prepared dataset was used to train a regression model that learns patterns between property features and their selling prices.

7.1 Selection of Features and Target Variable

The dataset features were divided into input variables (X) and the target variable (y). The input variables represent property characteristics, while SalePrice is the target variable that the model predicts.

Only records with known selling prices were used for supervised model training and testing.

7.2 Training and Testing Data

The labelled dataset was divided into training and testing sets using the train_test_split() function from Scikit-learn.

The training dataset was used to teach the model, while the testing dataset was used to evaluate its performance on unseen records. This separation helps assess how well the model generalizes to new data.

6 — Split Data into Training and Testing Sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (1168, 12)
X_test: (292, 12)
y_train: (1168,)
y_test: (292,)
```

7.3 Data Preprocessing

Data preprocessing was performed to prepare the input features for machine learning. Numerical and categorical features were handled according to their data types, and missing values were addressed using suitable techniques.

Categorical features were converted into a numerical representation where required. These steps help ensure that the data is in a suitable format for the selected regression algorithm.

7.4 Random Forest Regressor

The Random Forest Regressor was used for house price prediction. It is an ensemble machine learning algorithm that combines predictions from multiple decision trees to estimate a continuous numerical value.

The project also explored an improved Random Forest model by adjusting parameters such as the number of trees, maximum tree depth, and minimum samples per leaf.

9 — Train Random Forest Regression Model

```python
from sklearn.ensemble import RandomForestRegressor

model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ))
])

model.fit(X_train, y_train)

print("Model training completed successfully!")
```

```
Model training completed successfully!
```

7.5 Model Training and Prediction

The model was trained using the prepared training dataset. After training, it generated predictions for the test dataset. The predicted prices were then compared with actual selling prices to evaluate model performance.

The trained model can also be used to estimate the selling price of a house when its required features are provided.

```python
# Select one house from the test dataset
sample_house = X_test.iloc[[0]]

# Predict its price using the improved model
predicted_price = improved_model.predict(sample_house)[0]

# Get the actual price for comparison
actual_price = y_test.iloc[0]

print("Predicted House Price:", round(predicted_price, 2))
print("Actual House Price:", round(actual_price, 2))
print("Absolute Prediction Error:", round(abs(actual_price - predicted_price), 2))
```

```
Predicted House Price: 154170.69
Actual House Price: 154500.0
Absolute Prediction Error: 329.31
```

8. MODEL EVALUATION AND PERFORMANCE ANALYSIS

Model evaluation is the process of measuring how accurately a machine learning model predicts house prices. In this project, the trained regression model was evaluated by comparing predicted selling prices with actual selling prices from the testing dataset.

8.1 Evaluation Metrics

The following metrics were used to assess the regression model's performance.

Mean Absolute Error (MAE): MAE measures the average absolute difference between actual and predicted house prices. A lower MAE indicates that predictions are closer to actual values on average.

Root Mean Squared Error (RMSE): RMSE measures prediction errors while giving greater weight to larger errors. A lower RMSE generally indicates better predictive performance.

R-squared (R^2) Score: R^2 measures how well the model explains the variation in the target variable. A value closer to 1 indicates a better fit, although performance must also be considered in the context of the dataset.

2 — Evaluate Model Performance

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Mean Absolute Error (MAE):", round(mae, 2))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
print("R² Score:", round(r2, 4))
```

```
Mean Absolute Error (MAE): 22945.36
Root Mean Squared Error (RMSE): 37063.48
R² Score: 0.8209
```

8.2 Random Forest Model Performance

The Random Forest Regressor was evaluated using the testing dataset. The MAE, RMSE, and R^2 score were calculated to understand the accuracy of its predictions.

These metrics provide different perspectives on prediction errors and the model's ability to capture patterns in house prices.

8.3 Model Improvement

An improved Random Forest configuration was explored by adjusting model parameters. Its performance was compared with the initial model using the same evaluation metrics.

Comparing model configurations helps determine whether parameter changes improve predictive performance on the testing data.

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

improved_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=300,
        max_depth=20,
        min_samples_leaf=2,
        random_state=42,
        n_jobs=-1
    ))
])

improved_model.fit(X_train, y_train)

improved_pred = improved_model.predict(X_test)

improved_mae = mean_absolute_error(y_test, improved_pred)
improved_rmse = np.sqrt(mean_squared_error(y_test, improved_pred))
improved_r2 = r2_score(y_test, improved_pred)

print("Improved Model MAE:", round(improved_mae, 2))
print("Improved Model RMSE:", round(improved_rmse, 2))
print("Improved Model R²:", round(improved_r2, 4))
```

```
Improved Model MAE: 22986.02
Improved Model RMSE: 36580.73
Improved Model R²: 0.8255
```

8.4 Categorical-Only Model Comparison

A categorical-only model was also evaluated to examine the predictive value of categorical property features without relying on numerical features.

Its evaluation results were compared with those of the improved Random Forest model to understand the limitations of using categorical information alone.

```
Categorical-Only Model Results
Mean Absolute Error (MAE): 47105.94
R2 Score: 0.3069
```

8.5 Evaluation Summary

The evaluation results help assess the usefulness of the trained model for house price prediction. MAE and RMSE describe the magnitude of prediction errors, while R^2 indicates how well the model explains variations in selling prices. Together, these metrics support an informed assessment of model performance.

9. SAMPLE HOUSE PRICE PREDICTION

9.1 Prediction Process

After training and evaluating the Random Forest Regressor, the model was used to estimate the selling price of a sample house. The prediction was generated using the property features required by the trained model.

```
# Select one house from the test dataset
sample_house = X_test.iloc[[0]]

# Predict its price using the improved model
predicted_price = improved_model.predict(sample_house)[0]

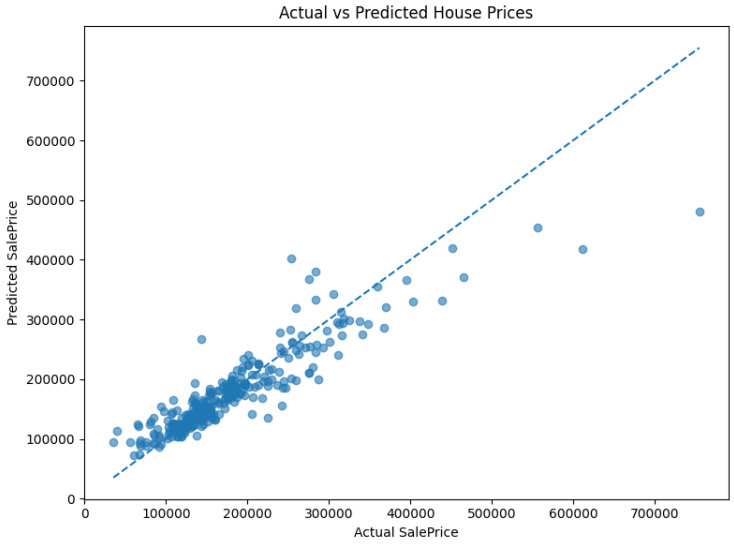
# Get the actual price for comparison
actual_price = y_test.iloc[0]

print("Predicted House Price:", round(predicted_price, 2))
print("Actual House Price:", round(actual_price, 2))
print("Absolute Prediction Error:", round(abs(actual_price - predicted_price), 2))
```

```
Predicted House Price: 154170.69
Actual House Price: 154500.0
Absolute Prediction Error: 329.31
```

9.2 Predicted and Actual Price Comparison

The predicted selling price was compared with the actual selling price to examine the model's performance on a sample property. This comparison provides a practical demonstration of the model's application to house price estimation.



9.3 Practical Application

The project demonstrates how machine learning can support house price estimation by analyzing property characteristics. Such predictions can provide an approximate reference for buyers, sellers, and property analysts. However, actual market prices may vary depending on factors not included in the dataset.

10. CONCLUSION

The House Price Prediction project demonstrates the application of machine learning techniques to estimate residential property prices using Python. The project involved loading and understanding the dataset, analyzing missing values, performing Exploratory Data Analysis (EDA), and examining the relationships between housing features and selling prices.

A Random Forest Regressor was implemented using Scikit-learn to predict house prices. The model was trained on labelled data and evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the R-squared (R^2) score. An improved model configuration and a categorical-only model were also explored to understand differences in predictive performance.

The project provided practical experience in data preprocessing, data visualization, regression modelling, and model evaluation using Google Colab. It also demonstrated how machine learning can be applied to a real-world estimation problem.

Overall, this project strengthened my understanding of the machine learning workflow, from data preparation to prediction, and highlighted the importance of evaluating model performance before using predictions in practical applications.

11. LIMITATIONS AND FUTURE SCOPE

11.1 Project Limitations

Although the House Price Prediction project demonstrates the use of machine learning for price estimation, it has certain limitations.

1. **Limited Features:** The dataset contains a selected set of housing features. Other factors, such as location details, nearby facilities, market demand, and economic conditions, may also influence property prices.
2. **Missing Data:** Some dataset columns contain missing values. These values require appropriate handling to avoid affecting the quality of analysis and predictions.
3. **Prediction Errors:** The model's predictions may differ from actual selling prices because of variations in property characteristics and market conditions.
4. **Dataset Dependency:** The model learns patterns from the available dataset. Its performance on houses from different locations or market conditions may vary.
5. **Model Limitations:** The Random Forest Regressor may not capture every factor affecting house prices, and its performance depends on the quality of the data and selected features.

11.2 Future Scope

The project can be improved in several ways:

1. **Additional Features:** Include more housing information, such as location, number of rooms, amenities, and nearby facilities, when suitable data is available.
2. **Model Comparison:** Evaluate additional regression algorithms, such as Linear Regression, Decision Tree Regression, and Gradient Boosting, to compare their performance.
3. **Hyperparameter Tuning:** Use systematic tuning techniques to identify suitable model parameters and potentially improve prediction accuracy.
4. **Improved Data Processing:** Apply appropriate methods for handling missing values, unusual observations, and categorical features.
5. **Web Application:** Develop a simple web application that allows users to enter property details and receive an estimated selling price.
6. **Regular Updates:** Retrain the model with newer housing data when available to help it reflect changing market patterns.

11.3 Overall Scope

With additional features, suitable model tuning, and a user-friendly interface, this project could be developed into a more practical house price estimation tool. Further testing on relevant datasets would be necessary before using it for real-world property decisions.